APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

KTU CST414 — DEEP LEARNING

MODULE 5 HANDWRITTEN LECTURE NOTES

B.Tech Computer Science and Engineering · Scheme 2019

Course Code: CST414	Credits: 3
Course Name: Deep Learning	Lecture Hours: 36
Faculty In-Charge: Jane Rose	Department: Computer Science & Engineering
Design: High-Contrast Printable Notebook	Font: Authentic Chalk & Kalam Ink

Prepared for students and faculty for classroom lectures, print study, and university examination preparation.

COURSE LECTURE INDEX & MAPPING

Day	Topic ID	Lecture Title	Module	CO	Date
#28	S.1	Deep Learning Applications: Computer Vis...	MS	COS	07-Nov-2025
#29	S.2	Deep Learning Applications: Speech Recog...	MS	COS	10-Nov-2025
#30	S.3	Deep Learning Applications: Natural Lang...	MS	COS	12-Nov-2025
#31	S.4	Common Word Embedding: CBOW and Word2Vec	MS	COS	14-Nov-2025
#32	S.5	Common Word Embedding: Global Vectors fo...	MS	COS	17-Nov-2025
#33	S.6	Current Research Areas: Autoencoders and...	MS	COS	19-Nov-2025
#34	S.7	Current Research Areas: Representation L...	MS	COS	21-Nov-2025
#35	S.8	Current Research Areas: Boltzmann Machin...	MS	COS	24-Nov-2025
#36	S.9	Current Research Areas: Deep Belief Netw...	MS	COS	26-Nov-2025

LECTURE #28 · MODULE 5 (Topic 5.1)

Date: 07-Nov-2025
Period: Period 1

Title: Deep Learning Applications: Computer Vision

CO: CO5 Bloom's: Understand / Apply

Prerequisites: CNN fundamentals, Convolutions, Pooling, Day 18 & 21 concepts
Key Terminology: Computer Vision, Object Detection, Two-Stage Detectors, Single-Stage Detectors (YOLO), Intersection over Union (IoU), Non-Maximum Suppression (NMS), Mean Average Precision (mAP)

Computer Vision Taxonomy

Deep learning has transformed computer vision across four primary task tiers: 1. Image Classification: Predict a single discrete category label for the entire image (e.g. ResNet on ImageNet 1k). 2. Object Localization: Predict class label AND a single bounding box [x_min, y_min, x_max, y_max]. 3. Object Detection: Identify and localize ALL object instances across multiple categories with multiple bounding boxes. 4. Semantic & Instance Segmentation: Pixel-level dense category assignment (Semantic) or per-object mask extraction (Instance, e.g. Mask R-CNN).

Object Detection Architectures

1. Two-Stage Detectors (Faster R-CNN, Ren et al., 2015): - Stage 1: Region Proposal Network (RPN) slides across convolutional feature maps to generate candidate object proposals with anchor boxes. - Stage 2: RoI Pooling/Align extracts fixed-size feature vectors per proposal for classification and bounding box regression. - Advantage: High precision and small object recall. 2. One-Stage Detectors (YOLO - Redmon et al., 2016; SSD): - Divides image into an S x S spatial grid. Each grid cell directly predicts B bounding boxes, objectness confidences, and C class probabilities in a single forward pass. - Advantage: Ultra-fast real-time inference (45 to 150+ fps), enabling live edge deployment.

Evaluation Metrics

1. Intersection over Union (IoU): IoU = Area(Box_pred cap Box_gt) / Area(Box_pred cup Box_gt) A prediction is counted as True Positive if IoU >= threshold (typically 0.50 or 0.75). 2. Non-Maximum Suppression (NMS): Sorts candidate boxes by confidence score and iteratively suppresses lower-scoring overlapping boxes with IoU > threshold to eliminate redundant detections. 3. Mean Average Precision (mAP): Computes the area under the Precision-Recall curve for each class across IoU thresholds, averaged across all categories.

Exam Important Points

- Contrast Two-Stage (Faster R-CNN) with One-Stage (YOLO) detectors.
- Mathematical formulation and manual calculation of Intersection over Union (IoU).
- Non-Maximum Suppression (NMS) algorithm steps.
- Definitions of Precision, Recall, and Mean Average Precision (mAP).

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Manual Intersection over Union (IoU) Calculation

Statement: Two bounding boxes are defined on an image grid: Ground Truth Box A: [x1 = 20, y1 = 20, x2 = 80, y2 = 80] Predicted Box B: [x1 = 40, y1 = 40, x2 = 100, y2 = 100] Calculate: (a) Area of Box A, (b) Area of Box B, (c) Area of Intersection, (d) Area of Union, and (e) The exact Intersection over Union (IoU).

Given: Box A = [20, 20, 80, 80], Box B = [40, 40, 100, 100].

Governing Formula: Area = (x2 - x1) * (y2 - y1); Inter = max(0, x_right - x_left) * max(0, y_bottom - y_top); Union = Area_A + Area_B - Inter; IoU = Inter / Union

Step-by-Step Calculation:

Step 1: Compute Area of Box A: Width_A = 80 - 20 = 60; Height_A = 80 - 20 = 60. Area_A = 60 * 60 = 3,600 sq pixels.

Step 2: Compute Area of Box B: Width_B = 100 - 40 = 60; Height_B = 100 - 40 = 60. Area_B = 60 * 60 = 3,600 sq pixels.

Step 3: Compute Intersection Coordinates: x_left = max(20, 40) = 40. y_top = max(20, 40) = 40. x_right = min(80, 100) = 80. y_bottom = min(80, 100) = 80. Inter_width = 80 - 40 = 40; Inter_height = 80 - 40 = 40. Area_Inter = 40 * 40 = 1,600 sq pixels.

Step 4: Compute Area of Union: Area_Union = Area_A + Area_B - Area_Inter = 3,600 + 3,600 - 1,600 = 5,600 sq pixels.

Step 5: Compute IoU: IoU = 1,600 / 5,600 = 16 / 56 = 2 / 7 = 0.2857.

[FINAL EVALUATED ANSWER: Area_A = 3600, Area_B = 3600, Inter = 1600, Union = 5600, IoU = 0.2857 (28.57%)]

Pedagogical Note: Since IoU = 0.2857 is below the standard 0.50 threshold, this prediction is classified as a False Positive in detection benchmark scoring.

Problem 2 (Advanced (KTU Exam Level)): Mean Average Precision (mAP) 11-Point Interpolation Calculation

Statement: An object detector outputs 5 ranked predictions for a class with 4 ground truth instances. Ranked detections have precision and recall values as follows: Rank 1: Precision = 1.00, Recall = 0.25 (TP) Rank 2: Precision = 0.50, Recall = 0.25 (FP) Rank 3: Precision = 0.67, Recall = 0.50 (TP) Rank 4: Precision = 0.50, Recall = 0.50 (FP) Rank 5: Precision = 0.60, Recall = 0.75 (TP) Using the standard Pascal VOC 11-point interpolation method at recall levels r in {0.0, 0.1, 0.2, ..., 1.0} where p_interp(r) = max_{r_tilde >= r} p(r_tilde): Calculate the Average Precision (AP).

Given: 4 ground truth objects, 5 ranked detections with (P, R) pairs.

Governing Formula: AP_11 = (1 / 11) * sum_{r in {0.0, 0.1, ..., 1.0}} p_interp(r)

Step-by-Step Calculation:

Step 1: Maximum precision values achievable for r_tilde >= r: For r in [0.0, 0.25]: Max precision = max(1.00, 0.67, 0.60) = 1.00. For r in (0.25, 0.50]: Max precision = max(0.67, 0.60) = 0.67. For r in (0.50, 0.75]: Max precision = 0.60. For r in (0.75, 1.00]: Since max recall reached is 0.75, precision = 0.00.

Step 2: Evaluate 11 recall thresholds: r = 0.0: p = 1.00 r = 0.1: p = 1.00 r = 0.2: p = 1.00 r = 0.3: p = 0.67 r = 0.4: p = 0.67 r = 0.5: p = 0.67 r = 0.6: p = 0.60 r = 0.7: p = 0.60 r = 0.8: p = 0.00 r = 0.9: p = 0.00 r = 1.0: p = 0.00.

Step 3: Sum interpolated precisions: Sum = 1.00*3 + 0.67*3 + 0.60*2 + 0.00*3 = 3.00 + 2.01 + 1.20 = 6.21.

Step 4: Compute Average Precision: AP = 6.21 / 11 = 0.5645.

[FINAL EVALUATED ANSWER: 11-point interpolated Average Precision AP = 0.5645 (56.45%)]

Pedagogical Note: The 11-point interpolation method smooths out precision fluctuations, penalizing the model for failing to reach 100% recall (precision drops to 0 at r >= 0.8).

Real-World Industry Case (Automotive Safety & Self-Driving Vehicles): Autonomous Highway Collision Avoidance & Emergency Braking

Pipeline Workflow: Triple front-facing camera array streams 60 fps 4K video -> Real-time YOLOv8 detector localizes oncoming vehicles, lane dividers, and pedestrians -> IoU tracker associates objects across frames -> Triggers automatic emergency braking if collision time < 1.2 seconds.

Industrial Impact & Benefit: Slashes forward vehicle collision accidents by 73% in poor visibility rain and fog conditions.

Key Oral Viva Voce Checkpoints

Q1: What is the primary difference between image classification and object detection?

Ans: Classification identifies what object is in the image; detection identifies what objects are present AND where they are located using bounding box coordinates.

Q2: What is Intersection over Union (IoU)?

Ans: The ratio of the area of overlap between the predicted box and ground truth box to the total area of their union.

Q3: What is the typical IoU threshold for counting a detection as a True Positive?

Ans: Typically IoU >= 0.50 (or 0.75 for strict benchmark scoring like COCO mAP@[.5:.95]).

Q4: What is Non-Maximum Suppression (NMS)?

Ans: A post-processing algorithm that eliminates redundant, overlapping bounding boxes by iteratively keeping the highest scoring box and discarding neighboring boxes with IoU above a threshold.

Q5: What is the key architectural difference between Faster R-CNN and YOLO?

Ans: Faster R-CNN is a two-stage detector (Region Proposal Network followed by classifier); YOLO is a single-stage detector that predicts boxes and classes directly from feature grids in one forward pass.

LECTURE #29 · MODULE 5 (Topic 5.2)

Date: 10-Nov-2025
Period: Period 2

Title: Deep Learning Applications: Speech Recognition

CO: CO5 Bloom's: Understand / Apply

Prerequisites: Audio Signal Processing, Recurrent Networks, CTC loss intuition, Day 23 & 26 concepts
Key Terminology: Automatic Speech Recognition (ASR), Mel Frequency Cepstral Coefficients (MFCC), Mel Filterbank, Connectionist Temporal Classification (CTC), Blank Token, Word Error Rate (WER)

Speech Representation

Speech is an analog acoustic pressure wave sampled at high frequencies (16 kHz = 16,000 samples/sec). Acoustic Feature Extraction: 1. Short-Time Fourier Transform (STFT): Apply a Hamming window of 25ms with 10ms hop size to extract local frequency spectrum. 2. Mel Scale Filterbank: Warps linear frequency into the perceptual Mel scale, matching human ear sensitivity (dense filters at low frequencies, wide filters at high frequencies): m = 2595 * log10(1 + f / 700) Yields an 80-channel log-Mel spectrogram tensor (Time x 80).

Connectionist Temporal Classification

Challenge: Audio frames T (e.g. 1,000 frames) far exceed target character length U (e.g. 50 characters), and exact frame-level alignments are unknown. CTC Solution (Graves et al., 2006): 1. Introduces a special 'Blank' token epsilon into the vocabulary: V_prime = V cup {epsilon}. 2. The network outputs a probability distribution over V_prime at every audio frame t. 3. Collapse Mapping B: Collapses consecutive duplicate characters and then removes blank tokens: B('a' 'a' epsilon 'b' 'b') = 'a' 'b' B('c' epsilon 'c') = 'c' 'c' (blank allows generating identical consecutive letters!). 4. Loss Function: Sums conditional probabilities across all valid alignment paths pi that map to target y: P(y | x) = sum_{pi in B^(-1)(y)} product_{t=1}^T P(pi_t | x) L_CTC = - ln P(y | x), optimized via dynamic programming (forward-backward algorithm).

Evaluation Metrics

Word Error Rate (WER): WER = (S + D + I) / N where S is Substitutions, D is Deletions, I is Insertions, and N is total ground truth words.

Exam Important Points

- The role of the Mel scale filterbank in acoustic preprocessing.
- Why CTC loss is necessary for unaligned audio-to-text sequence modeling.
- How the CTC blank token enables spelling repeated letters (e.g. 'hello').
- Formulation and manual calculation of Word Error Rate (WER).

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Word Error Rate (WER) Calculation

Statement: An Automatic Speech Recognition (ASR) system transcribes a recorded meeting sentence. The reference transcript contains N = 10 words: Reference: 'the weather in southern kerala is hot and humid today' Hypothesis: 'the weather in kerala is very hot humid today' Identify the number of Substitutions (S), Deletions (D), and Insertions (I), and calculate the Word Error Rate (WER).

Given: Reference: 10 words. Hypothesis text as given.

Governing Formula: WER = (S + D + I) / N

Step-by-Step Calculation:

Step 1: Align reference and hypothesis words: 'the' -> 'the' (Match) 'weather' -> 'weather' (Match) 'in' -> 'in' (Match) 'southern' -> [DELETED] (Deletion: D = 1) 'kerala' -> 'kerala' (Match) 'is' -> 'is' (Match) [NONE] -> 'very' (Insertion: I = 1) 'hot' -> 'hot' (Match) 'and' -> [DELETED] (Deletion: D = 2) 'humid' -> 'humid' (Match) 'today' -> 'today' (Match).

Step 2: Total error counts: Substitutions (S) = 0. Deletions (D) = 2 ('southern', 'and'). Insertions (I) = 1 ('very'). Total Errors = 0 + 2 + 1 = 3 errors.

Step 3: Calculate WER: WER = (S + D + I) / N = 3 / 10 = 0.3000 (30.0%).

[FINAL EVALUATED ANSWER: S = 0, D = 2, I = 1. Total Errors = 3. Word Error Rate (WER) = 30.0%.]

Pedagogical Note: Two deleted words and one hallucinated insertion yield an overall word error rate of 30% against the 10-word reference.

Problem 2 (Advanced (KTU Exam Level)): Mel Frequency Calculation from Hertz Audio Pitch

Statement: The Mel scale formula relating acoustic physical frequency f in Hertz (Hz) to perceived psychological pitch m in Mels is m = 2595 * log10(1 + f / 700). (a) Calculate the Mel pitch for frequencies f1 = 300 Hz, f2 = 1000 Hz, and f3 = 4000 Hz, (b) Calculate the delta in Hertz between f2 and f1 vs between f3 and f2, and (c) Compare the corresponding deltas on the Mel scale to illustrate human ear non-linearity.

Given: m = 2595 * log10(1 + f / 700). Frequencies: 300 Hz, 1000 Hz, 4000 Hz.

Governing Formula: m = 2595 * log10(1 + f / 700)

Step-by-Step Calculation:

Step 1: For f1 = 300 Hz: m1 = 2595 * log10(1 + 300 / 700) = 2595 * log10(1 + 0.42857) = 2595 * log10(1.42857) = 2595 * 0.15490 = 401.97 Mels.

Step 2: For f2 = 1000 Hz: m2 = 2595 * log10(1 + 1000 / 700) = 2595 * log10(1 + 1.42857) = 2595 * log10(2.42857) = 2595 * 0.38535 = 1000.00 Mels.

Step 3: For f3 = 4000 Hz: m3 = 2595 * log10(1 + 4000 / 700) = 2595 * log10(1 + 5.71429) = 2595 * log10(6.71429) = 2595 * 0.82700 = 2146.06 Mels.

Step 4: Hertz spacing vs Mel spacing: Delta_Hz(f2 - f1) = 1000 - 300 = 700 Hz -> Delta_Mel = 1000 - 401.97 = 598.03 Mels. Delta_Hz(f3 - f2) = 4000 - 1000 = 3000 Hz -> Delta_Mel = 2146.06 - 1000 = 1146.06 Mels. In Hertz, interval 2 is 3000/700 = 4.29x wider, but in Mel space it is only 1146/598 = 1.92x wider!

[FINAL EVALUATED ANSWER: m(300Hz) = 402.0 Mels, m(1000Hz) = 1000.0 Mels, m(4000Hz) = 2146.1 Mels.]

Pedagogical Note: Human hearing has high perceptual resolution at lower frequencies (under 1 kHz) and compressed logarithmic sensitivity at high frequencies, which the Mel filterbank preserves.

Real-World Industry Case (Public Safety & Telecommunications): Real-Time Emergency Dispatch Voice Call Transcription

Pipeline Workflow: Distressed citizen calls 911/112 -> Audio stream contains high acoustic background siren noise -> 80-band Mel filterbank fed into Conformer CTC ASR engine -> Transcribes street address and emergency details in sub-second latency.

Industrial Impact & Benefit: Enables emergency first responders to dispatch ambulances 45 seconds faster, saving critical lives.

Key Oral Viva Voce Checkpoints

Q1: What is the physical sampling rate of standard high-fidelity voice audio?

Ans: 16 kHz (16,000 samples per second) for speech recognition, or 44.1 kHz for studio music.

Q2: Why is the Mel scale preferred over linear frequency in speech recognition?

Ans: The Mel scale matches human auditory perception, which has high resolution for low frequencies and compressed sensitivity for high frequencies.

Q3: What is Connectionist Temporal Classification (CTC)?

Ans: A loss function designed for sequence tasks where the input sequence is longer than the output sequence and exact temporal alignments are unknown.

Q4: What is the function of the blank token epsilon in CTC?

Ans: To indicate silence/transitions and to allow the model to emit repeated characters (e.g. 'b l a n k' vs 'l o o k').

Q5: How does the CTC collapse function decode 'c c _ a a a _ t t'?

Ans: First it collapses consecutive duplicates: 'c _ a _ t'; then it removes blanks: 'cat'.

LECTURE #30 · MODULE 5 (Topic 5.3)

Date: 12-Nov-2025
Period: Period 4

Title: Deep Learning Applications: Natural Language Processing

CO: CO5 Bloom's: Understand / Apply

Prerequisites: Seq2Seq, Word Embeddings intuition, Cross-Entropy Loss, Day 24 concepts
Key Terminology: Natural Language Processing (NLP), Autoregressive Language Modeling, Perplexity (PPL), Sentiment Classification, Tokenization, Cross-Entropy Loss, Transformer Transition

Nlp Task Taxonomy

Deep learning models natural language across diverse granularities: 1. Document / Sentence-Level Classification: - Sentiment Analysis: Input text -> Output sentiment label (Positive, Neutral, Negative). - Topic Categorization: Input news article -> Output genre (Sports, Tech, Politics). 2. Token-Level Sequence Tagging: - Named Entity Recognition (NER): Tagging Person, Organization, Location per token. - Part-of-Speech (POS) Tagging: Noun, Verb, Adjective. 3. Autoregressive Language Modeling: - Decomposes the joint probability of a sequence of tokens w_1, ..., w_T using the chain rule of probability: P(w_1, w_2, ..., w_T) = product_{t=1}^T P(w_t | w_1, w_2, ..., w_{t-1}) - Foundation for modern generative LLMs (GPT, Llama, Gemini).

Perplexity Metric

Perplexity (PPL) is the standard quantitative metric for evaluating language models. It is defined as the exponentiated average negative log-likelihood per token: PPL = exp(- (1 / T) * sum_{t=1}^T ln P(w_t | w_{<t})) = 2^(Cross_Entropy_Loss_in_bits) Interpretation: Perplexity represents the weighted average branching factor of the language model — the effective number of equally likely words the model is uncertain between at each prediction step. Lower perplexity indicates superior predictive certainty!

Exam Important Points

- Chain rule decomposition of joint sequence probability in language modeling.
- Mathematical formulation and physical interpretation of Perplexity (PPL).
- Deep learning architecture for Sentiment Analysis using Bidirectional LSTMs.
- Limitations of recurrent NLP models (sequential O(T) latency, lack of parallel training).

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Perplexity Calculation from Cross-Entropy Loss

Statement: A neural language model evaluates a test document containing T = 1,000 words. The total negative log-likelihood of the document under natural logarithm (base e) is sum_{t=1}^T - ln P(w_t | w_{<t}) = 3,500 nats. (a) Calculate the average cross-entropy loss per token in nats, (b) Calculate the Perplexity (PPL) of the model, and (c) If a competing model achieves Perplexity PPL = 20, calculate its average loss per token.

Given: T = 1,000 words, Total Loss = 3,500 nats.

Governing Formula: Avg_Loss = Total_Loss / T; PPL = exp(Avg_Loss); Avg_Loss = ln(PPL)

Step-by-Step Calculation:

Step 1: Compute average cross-entropy loss per token: Avg_Loss = 3,500 / 1,000 = 3.50 nats/token.

Step 2: Calculate Perplexity: PPL = e^(3.50) = 33.115.

Step 3: Competing model with PPL = 20: Loss = ln(20) = 2.9957 nats/token.

Step 4: Comparison: The competing model has lower perplexity (20.0 vs 33.1) and lower loss (3.00 vs 3.50), indicating stronger language modeling ability.

[FINAL EVALUATED ANSWER: Average Loss = 3.50 nats, Perplexity PPL = 33.12. Competing Model Loss = 3.00 nats.]

Pedagogical Note: A perplexity of 33.12 means the language model is effectively as uncertain as guessing from a uniformly distributed vocabulary of 33 words at each step.

Problem 2 (Advanced (KTU Exam Level)): Sentiment Classifier Binary Cross-Entropy and Gradient Update

Statement: A sentiment analysis model processes a movie review and outputs probability y_hat = 0.85 for Positive class (ground truth y = 1.0). The output layer computes y_hat = sigma(z) where z = w^T * h + b. Current weight is w_1 = 0.50, hidden activation is h_1 = 0.80, and learning rate is eta = 0.10. Loss is Binary Cross-Entropy L = - [y*ln(y_hat) + (1-y)*ln(1-y_hat)]. (a) Calculate the loss value, (b) Compute the gradient dL/dz and dL/dw_1, and (c) Compute the updated weight w_1_new.

Given: y = 1.0, y_hat = 0.85, w_1 = 0.50, h_1 = 0.80, eta = 0.10.

Governing Formula: L = - ln(y_hat); dL/dz = y_hat - y; dL/dw_1 = (dL/dz) * h_1; w_new = w - eta * (dL/dw)

Step-by-Step Calculation:

Step 1: Compute Binary Cross-Entropy Loss: L = - [1.0 * ln(0.85) + 0] = - (-0.1625) = 0.1625.

Step 2: Compute gradient with respect to logit z: dL/dz = y_hat - y = 0.85 - 1.00 = -0.15.

Step 3: Compute gradient with respect to weight w_1: dL/dw_1 = (dL/dz) * h_1 = (-0.15) * 0.80 = -0.120.

Step 4: Update weight: w_1_new = w_1 - eta * (dL/dw_1) = 0.50 - 0.10 * (-0.120) = 0.50 + 0.012 = 0.5120.

[FINAL EVALUATED ANSWER: Loss L = 0.1625, Gradient dL/dw_1 = -0.120, Updated weight w_1_new = 0.5120]

Pedagogical Note: Because the model underpredicted slightly (0.85 vs 1.0), the negative gradient increased weight w_1 to reinforce positive sentiment confidence in future passes.

Real-World Industry Case (Fintech & Quantitative Regulatory Analysis): Automated Financial SEC 10-K Filing Risk Assessment

Pipeline Workflow: Public corporate annual 10-K financial disclosures (100+ pages) -> Deep bidirectional NLP model scans Risk Factors section -> Categorizes litigation, supply chain, and cyber risks -> Generates portfolio risk scores.

Industrial Impact & Benefit: Enables asset managers to detect emerging corporate insolvency risks weeks before quarterly earnings announcements.

Key Oral Viva Voce Checkpoints

Q1: What is an Autoregressive Language Model?

Ans: A model that predicts the probability of the next word conditioned on all previously generated context words: P(w_t | w_1, ..., w_{t-1}).

Q2: What is Perplexity in language modeling?

Ans: The exponentiated cross-entropy loss, representing the effective branching factor (number of uncertain choices) per token.

Q3: What is the relationship between Perplexity and Cross-Entropy Loss?

Ans: PPL = exp(Loss_nats) or PPL = 2^(Loss_bits). Lower perplexity indicates superior predictive performance.

Q4: What is Named Entity Recognition (NER)?

Ans: A token-level sequence classification task that identifies mentions of predefined real-world entities (persons, locations, organizations) in text.

Q5: How does subword tokenization (Byte-Pair Encoding, BPE) resolve the Out-of-Vocabulary (OOV) problem?

Ans: By decomposing rare and unseen words into frequent subword units (e.g. 'unpredictable' -> 'un' + 'predict' + 'able'), allowing any arbitrary string to be represented.

LECTURE #31 · MODULE 5 (Topic 5.4)

Date: 14-Nov-2025
Period: Period 1

Title: Common Word Embedding: CBOW and Word2Vec

CO: CO5 Bloom's: Understand / Apply

Prerequisites: Vector representations, Softmax, Cross-Entropy, Day 3 concepts
Key Terminology: Word Embeddings, One-Hot Encoding, Word2Vec, Continuous Bag of Words (CBOW), Continuous Skip-Gram, Negative Sampling (SGNS), Vector Arithmetic

One Hot Vs Distributed Embeddings

One-Hot Encoding: - Represents each word as a sparse binary vector of vocabulary size |V| (e.g. 100,000 dimensions). - Fatal Flaw: All word vectors are orthogonal: v_cat^T * v_dog = 0. No notion of semantic distance or similarity! Distributed Word Embeddings: - Maps words to dense, low-dimensional continuous vectors in R^d (d = 100 to 300). - Words appearing in similar contexts acquire similar geometric representations (Distributional Hypothesis: Firth, 1957).

Word2Vec Cbow Vs Skipgram

Tomas Mikolov et al. (Google, 2013) introduced Word2Vec with two complementary architectures: 1. Continuous Bag-of-Words (CBOW): - Objective: Predict a target center word w_t given context words in a window: {w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}}. - Mechanism: Averages context word vectors: v_context = (1 / 2C) * sum_{c in context} v_c. - Characteristics: Faster training, slightly better on frequent words. 2. Continuous Skip-Gram: - Objective: Given a single center word w_t, predict surrounding context words w_{t+j}. - Characteristics: Slower training, but superior representations for rare words. Skip-Gram with Negative Sampling (SGNS): Evaluating standard Softmax requires summing exp(v^T * v') over the entire vocabulary |V| (costly O(|V|)). Negative Sampling replaces multiclass classification with k binary logistic regressions: L_SGNS = - ln sigma(v'_{w_O}^T * v_{w_I}) - sum_{i=1}^k E_{w_i ~ P_n(w)} [ln sigma(- v'_{w_i}^T * v_{w_I})] where negative words are drawn from unigram distribution raised to the 3/4 power: P_n(w) propto f(w)^(0.75).

Vector Arithmetic

Linear compositionality: Word vectors capture semantic and grammatical analogies via simple vector arithmetic: vec('King') - vec('Man') + vec('Woman') approx vec('Queen') vec('Paris') - vec('France') + vec('Germany') approx vec('Berlin')

Exam Important Points

- Architectural diagrams comparing CBOW (context -> target) vs Skip-Gram (target -> context).
- Why negative sampling is used instead of standard Softmax (computational complexity).
- Unigram distribution exponent 3/4 rationale for negative sampling.
- Semantic vector arithmetic: King - Man + Woman = Queen.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Cosine Similarity Calculation Between Word Vectors

Statement: Two word embedding vectors in 3-dimensional semantic space are given as: v_doctor = [0.8, 0.6, 0.0]^T v_hospital = [0.6, 0.8, 0.0]^T v_banana = [0.0, 0.1, 0.9]^T Calculate: (a) Euclidean norms of all three vectors, (b) Cosine similarity between 'doctor' and 'hospital', and (c) Cosine similarity between 'doctor' and 'banana'.

Given: v_doc = [0.8, 0.6, 0], v_hosp = [0.6, 0.8, 0], v_ban = [0, 0.1, 0.9].

Governing Formula: ||v|| = sqrt(sum v_i^2); cos_sim(u, v) = (u . v) / (||u|| * ||v||)

Step-by-Step Calculation:

Step 1: Compute Euclidean norms: ||v_doc|| = sqrt(0.8^2 + 0.6^2 + 0) = sqrt(0.64 + 0.36) = sqrt(1.0) = 1.00. ||v_hosp|| = sqrt(0.6^2 + 0.8^2 + 0) = sqrt(0.36 + 0.64) = sqrt(1.0) = 1.00. ||v_ban|| = sqrt(0 + 0.1^2 + 0.9^2) = sqrt(0.01 + 0.81) = sqrt(0.82) = 0.9055.

Step 2: Compute dot products: v_doc . v_hosp = (0.8 * 0.6) + (0.6 * 0.8) + (0 * 0) = 0.48 + 0.48 = 0.96. v_doc . v_ban = (0.8 * 0) + (0.6 * 0.1) + (0 * 0.9) = 0.06.

Step 3: Compute Cosine Similarities: cos_sim(doc, hosp) = 0.96 / (1.00 * 1.00) = 0.9600 (96.0% similarity). cos_sim(doc, ban) = 0.06 / (1.00 * 0.9055) = 0.0663 (6.63% similarity).

[FINAL EVALUATED ANSWER: cos(doc, hosp) = 0.9600 (Strongly Related). cos(doc, ban) = 0.0663 (Unrelated).]

Pedagogical Note: Cosine similarity successfully captures high semantic relatedness between clinical concepts while showing near-zero alignment with unrelated entities.

Problem 2 (Advanced (KTU Exam Level)): Skip-Gram Computational Complexity: Full Softmax vs Negative Sampling

Statement: A corpus has vocabulary size |V| = 200,000 words. Word embedding dimension is d = 300. (a) Calculate the multiplication operations required per training step to evaluate full Softmax over the vocabulary, (b) Calculate the operations required under Negative Sampling with k = 5 negative samples, and (c) Determine the exact computational speedup factor achieved by SGNS.

Given: |V| = 200,000, d = 300, k = 5 negative samples.

Governing Formula: Full Softmax Ops = |V| * d (for logits) + |V| (for exp/sum); SGNS Ops = (1 + k) * d

Step-by-Step Calculation:

Step 1: Compute Full Softmax operations: Computing inner products for all vocabulary words: 200,000 * 300 = 60,000,000 multiplications. Exponentiation and normalizer sum: ~200,000 ops. Total Full Softmax ops approx 60,000,000 operations per word update.

Step 2: Compute Negative Sampling (SGNS) operations: SGNS evaluates 1 true positive pair + k negative pairs = 1 + 5 = 6 dot products. Total multiplications = 6 * 300 = 1,800 operations.

Step 3: Calculate computational speedup: Speedup = 60,000,000 / 1,800 = 33,333.33x (~33,333 times faster!).

[FINAL EVALUATED ANSWER: Full Softmax = 60M ops. SGNS = 1,800 ops (Over 33,000x faster!).]

Pedagogical Note: Negative sampling replaces a massive 60-million operation vocabulary sweep with just 1,800 simple multiplications, transforming Word2Vec from an intractable theoretical model into an ultra-fast algorithm trainable on billions of words in minutes.

Real-World Industry Case (Retail E-Commerce & Product Discovery): E-Commerce Semantic Search & Recommendation Engine

Pipeline Workflow: Customer searches 'running shoes' on shopping app -> Catalog items lack literal string match for 'sneakers' -> Word2Vec item embeddings compute high cosine similarity between 'running shoes' and 'sneakers' -> Recommends relevant footwear.

Industrial Impact & Benefit: Increases search conversion rates by 19% by overcoming the vocabulary mismatch problem.

Key Oral Viva Voce Checkpoints

Q1: What is the primary limitation of one-hot word representations?

Ans: High dimensionality, extreme sparsity, and complete lack of semantic distance or similarity between words (all one-hot vectors are orthogonal).

Q2: What is the Distributional Hypothesis in linguistics?

Ans: 'Words that occur in similar contexts tend to have similar meanings' (J.R. Firth, 1957).

Q3: What is the difference between CBOW and Skip-Gram?

Ans: CBOW predicts the center target word from surrounding context words; Skip-Gram predicts the surrounding context words given the center target word.

Q4: Why is standard Softmax computationally prohibitive in Word2Vec?

Ans: Because computing the denominator requires calculating inner products and exponentials over the entire vocabulary $|V|$ (hundreds of thousands of words) for every training step.

Q5: How does Negative Sampling (SGNS) solve the Softmax bottleneck?

Ans: It reformulates the task as binary logistic regression, updating only the positive word pair and k randomly chosen negative words, cutting complexity from $O(|V|)$ to $O(k)$.

LECTURE #32 · MODULE 5 (Topic 5.5)

Date: 17-Nov-2025
Period: Period 2

Title: Common Word Embedding: Global Vectors for Word Representation (GloVe)

CO: CO5 Bloom's: Understand / Apply

Prerequisites: Matrix Factorization, Word2Vec, Log-bilinear models, Day 31 concepts
Key Terminology: Global Vectors (GloVe), Co-occurrence Matrix, Co-occurrence Probability Ratio, Log-Bilinear Model, Weighted Least Squares, Weighting Function f(X_ij)

Glove Motivation

Prior to GloVe (2014), representation learning was divided into two distinct paradigms: 1. Global Matrix Factorization Methods (LSA): Leveraged global corpus statistics, but performed poorly on local word analogy tasks. 2. Local Context Window Methods (Word2Vec): Excellent on analogical reasoning, but failed to directly leverage vast global co-occurrence statistics. GloVe combines the benefits of both by directly fitting a log-bilinear model to the global word-word co-occurrence matrix X.

Mathematical Foundation And Cooccurrence Ratios

Let X_ij be the count of times word j appears in the context of word i in the corpus. Let P_ij = P(j | i) = X_ij / X_i be the probability that word j appears in the context of word i. The core insight of GloVe is that semantic meaning is captured by the RATIO of co-occurrence probabilities: Consider i = 'ice', j = 'steam', and probe words k: - k = 'solid': P(solid | ice) is high, P(solid | steam) is low -> Ratio P(solid|ice) / P(solid|steam) >> 1 - k = 'gas': P(gas | ice) is low, P(gas | steam) is high -> Ratio P(gas|ice) / P(gas|steam) << 1 - k = 'water': P(water | ice) is high, P(water | steam) is high -> Ratio approx 1 - k = 'fashion': P(fashion | ice) is low, P(fashion | steam) is low -> Ratio approx 1 GloVe models this ratio using dot products of word vectors: w_i^T * w_tilde_j + b_i + b_tilde_j = ln(X_ij)

Weighted Least Squares Objective

GloVe minimizes the weighted squared error between the dot product of word vectors and the log co-occurrence counts: J = sum_{i=1}^V sum_{j=1}^V f(X_ij) * (w_i^T * w_tilde_j + b_i + b_tilde_j - ln(X_ij))^2 Weighting Function f(X_ij): f(x) = { (x / x_max)^alpha if x < x_max { 1 if x >= x_max (typically x_max = 100, alpha = 0.75). Properties: 1. f(0) = 0: Ensures ln(0) is never evaluated, avoiding divergence on non-co-occurring pairs. 2. Non-decreasing: Frequent words carry appropriate statistical weight. 3. Saturating at 1: Prevents extremely common stop-words ('the', 'of') from dominating the objective.

Exam Important Points

- The co-occurrence probability ratio table (ice vs steam with solid, gas, water, fashion).
- Derivation of the log-bilinear equation: w_i^T * w_j + b_i + b_j = ln(X_ij).
- Full GloVe weighted least-squares objective function and properties of weighting function f(x).
- Comparison: Word2Vec vs GloVe.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): GloVe Weighting Function f(x) Calculation

Statement: The GloVe weighting function is defined as f(x) = (x / x_max)^alpha if x < x_max else 1.0, with x_max = 100 and alpha = 0.75. Calculate the weight f(x) assigned to: (a) A word pair with zero co-occurrences (x = 0), (b) A moderately frequent pair with x = 16 co-occurrences, (c) A frequent pair with x = 81 co-occurrences, and (d) An extremely frequent stop-word pair with x = 5,000 co-occurrences.

Given: x_max = 100, alpha = 0.75.

Governing Formula: f(x) = (x / 100)^0.75 if x < 100 else 1.0

Step-by-Step Calculation:

Step 1: For x = 0: f(0) = 0.0 (Eliminates log(0) singularity in objective function!).

Step 2: For x = 16: f(16) = (16 / 100)^0.75 = (0.16)^0.75 = (0.16^(3/4)) = (0.16^(1/4))^3 = (0.63245)^3 = 0.25298.

Step 3: For x = 81: f(81) = (81 / 100)^0.75 = (0.81)^0.75 = (0.81^(1/4))^3 = (0.94868)^3 = 0.8538.

Step 4: For x = 5,000: Since x = 5,000 >= x_max = 100, f(5,000) = 1.0 (Caps runaway stop-word weights).

[FINAL EVALUATED ANSWER: f(0) = 0.0, f(16) = 0.2530, f(81) = 0.8538, f(5000) = 1.0000]

Pedagogical Note: The weighting function smooths out frequency disparities, preventing zero-count crashes while preventing stop-word co-occurrences from dominating optimization.

Problem 2 (Advanced (KTU Exam Level)): GloVe Log-Bilinear Prediction and Loss Gradient

Statement: In a GloVe model, word vector w_i = [0.5, 0.4] and context vector w_tilde_j = [0.6, 0.2] with scalar biases b_i = 0.10 and b_tilde_j = 0.05. The observed co-occurrence count is X_ij = 2.0. The weight assigned by f(X_ij) is 0.05. (a) Calculate the dot product plus biases: z_ij = w_i^T * w_tilde_j + b_i + b_tilde_j, (b) Calculate the target log count ln(X_ij), (c) Compute the error e_ij = z_ij - ln(X_ij), and (d) Calculate the gradient of the loss with respect to bias b_i.

Given: w_i = [0.5, 0.4], w_tilde_j = [0.6, 0.2], b_i = 0.10, b_tilde_j = 0.05, X_ij = 2.0, f(X_ij) = 0.05.

Governing Formula: z = w_i^T * w_tilde_j + b_i + b_tilde_j; e = z - ln(X_ij); J_ij = f(X_ij) * e^2; dJ/db_i = 2 * f(X_ij) * e

Step-by-Step Calculation:

Step 1: Compute linear dot product plus biases: dot = (0.5 * 0.6) + (0.4 * 0.2) = 0.30 + 0.08 = 0.38. z_ij = 0.38 + 0.10 + 0.05 = 0.530.

Step 2: Compute target log count: ln(X_ij) = ln(2.0) = 0.69315.

Step 3: Compute prediction error: e_ij = z_ij - ln(X_ij) = 0.530 - 0.69315 = -0.16315.

Step 4: Compute gradient with respect to bias b_i: dJ/db_i = 2 * f(X_ij) * e_ij = 2 * 0.05 * (-0.16315) = 0.10 * (-0.16315) = -0.016315.

[FINAL EVALUATED ANSWER: z_ij = 0.530, Target ln(X_ij) = 0.6931, Error = -0.1631, Gradient dJ/db_i = -0.0163]

Pedagogical Note: Because the predicted log count (0.53) was smaller than the empirical count (0.69), the negative gradient will increase bias b_i to better match corpus statistics.

Real-World Industry Case (Legal Tech & Corporate Compliance): Legal Contract Discovery & Precedent Retrieval

Pipeline Workflow: Corporate legal repository contains 20 years of court filings -> GloVe embeddings trained on corpus capture domain-specific legal terminology associations ('plaintiff', 'tort', 'indemnity') -> Enables semantic case law search.

Industrial Impact & Benefit: Slashes paralegal legal discovery search time from 8 hours to 15 minutes per case.

Key Oral Viva Voce Checkpoints

Q1: What does the acronym GloVe stand for?

Ans: Global Vectors for Word Representation.

Q2: Who invented GloVe and in what year?

Ans: Jeffrey Pennington, Richard Socher, and Christopher D. Manning at Stanford University in 2014.

Q3: How does GloVe combine global matrix factorization and local context windows?

Ans: It constructs a global word-word co-occurrence matrix from local context windows and trains a log-bilinear regression model directly on this global matrix.

Q4: What is the core insight behind GloVe's relationship to word meaning?

Ans: The ratio of co-occurrence probabilities of two words with various probe words encodes semantic differences much more cleanly than raw probabilities.

Q5: What is the GloVe objective function?

*Ans: J = sum_{i,j} f(X_ij) * (w_i^T * w_tilde_j + b_i + b_tilde_j - ln(X_ij))^2.*

LECTURE #33 · MODULE 5 (Topic 5.6)

Date: 19-Nov-2025
Period: Period 4

Title: Current Research Areas: Autoencoders and Representation Learning

CO: CO5 Bloom's: Understand / Apply

Prerequisites: Encoder-Decoder mechanics, Unsupervised learning, Regularization, Day 2 & 14 concepts
Key Terminology: Autoencoder, Undercomplete Autoencoder, Latent Bottleneck, Reconstruction Loss, Sparse Autoencoder, Kullback-Leibler (KL) Divergence, Denoising Autoencoder (DAE), Contractive Autoencoder (CAE)

Autoencoder Architecture

An Autoencoder is a self-supervised neural network trained to reconstruct its own input: 1. Encoder: Maps input x in R^d to a latent code h in R^k: h = f(x) = sigma(W_e * x + b_e) 2. Decoder: Reconstructs input from latent code: r = g(h) = sigma(W_d * h + b_d) 3. Objective: Minimize reconstruction error L(x, g(f(x))) (e.g. MSE for continuous data, BCE for normalized pixels). If an autoencoder succeeds in learning g(f(x)) = x everywhere, it would simply memorize the identity function! To learn meaningful representations, constraints must be imposed.

Autoencoder Variants

1. Undercomplete Autoencoders: - Constrains latent dimension k < d (information bottleneck). - Theorem: If the encoder and decoder are linear and loss is MSE, the undercomplete autoencoder learns the exact same subspace as Principal Component Analysis (PCA)! - With non-linear activations, it learns a non-linear manifold generalization of PCA. 2. Sparse Autoencoders: - Overcomplete latent space (k > d), but penalizes average hidden unit activations rho_hat_j using KL divergence: L_sparse = L_recon + beta * sum_{j=1}^k KL(rho || rho_hat_j) where KL(rho || rho_hat) = rho * ln(rho / rho_hat) + (1 - rho) * ln((1 - rho) / (1 - rho_hat)). 3. Denoising Autoencoders (DAE, Vincent et al., 2008): - Corrupts input x with noise: x_tilde ~ q(x_tilde | x). - Trains the network to reconstruct the CLEAN input x from noisy x_tilde: L = L(x, g(f(x_tilde))). - Forces the model to learn the data manifold's score matching vector field pointing from noise back to the data manifold! 4. Contractive Autoencoders (CAE, Rifai et al., 2011): - Adds an explicit Frobenius norm penalty on the encoder's Jacobian matrix: || J_f(x) ||_F^2 = sum_i sum_j (del h_i / del x_j)^2. - Forces latent representations to be invariant (contractive) to small perturbations in the input space.

Exam Important Points

- KTU Exam Q10: Definition of autoencoder and one application (anomaly detection, dimensionality reduction).
- KTU Exam Q19(b): Merits and demerits of using Autoencoders in Computer Vision.
- Proof that linear autoencoder under MSE loss is equivalent to PCA.
- Formulation of Sparse Autoencoders and KL divergence sparsity penalty.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Kullback-Leibler (KL) Divergence Sparsity Penalty Calculation

Statement: In a sparse autoencoder, the target average activation is set to rho = 0.05. A hidden neuron j has computed average activation rho_hat_j = 0.10 across a training batch. Calculate: (a) The Kullback-Leibler divergence penalty KL(rho || rho_hat_j) for this neuron, and (b) The penalty if the neuron had perfectly satisfied the target (rho_hat = 0.05).

Given: Target rho = 0.05, Actual rho_hat = 0.10.

Governing Formula: KL(rho || rho_hat) = rho * ln(rho / rho_hat) + (1 - rho) * ln((1 - rho) / (1 - rho_hat))

Step-by-Step Calculation:

Step 1: Term 1: rho * ln(rho / rho_hat): rho / rho_hat = 0.05 / 0.10 = 0.50. ln(0.50) = -0.69315. Term 1 = 0.05 * (-0.69315) = -0.03466.

Step 2: Term 2: (1 - rho) * ln((1 - rho) / (1 - rho_hat)): 1 - rho = 1 - 0.05 = 0.95. 1 - rho_hat = 1 - 0.10 = 0.90. (1 - rho) / (1 - rho_hat) = 0.95 / 0.90 = 1.05556. ln(1.05556) = 0.05407. Term 2 = 0.95 * 0.05407 = 0.05137.

Step 3: Combine terms: KL(0.05 || 0.10) = -0.03466 + 0.05137 = 0.01671.

Step 4: If rho_hat = 0.05: KL(0.05 || 0.05) = 0.05 * ln(1) + 0.95 * ln(1) = 0.0000.

[FINAL EVALUATED ANSWER: KL(0.05 || 0.10) = 0.01671. Target penalty = 0.0000.]

Pedagogical Note: The positive KL divergence (0.0167) penalizes the neuron for being overly active (10% firing rate vs 5% target), pushing its bias negative to restore sparsity.

Problem 2 (Advanced (KTU Exam Level)): Reconstruction Error vs Dimensionality Reduction in Linear Autoencoder

Statement: An input vector x = [3.0, 4.0]^T is projected into a 1-dimensional latent subspace using linear encoder weight w = [0.8, 0.6]^T (unit norm ||w|| = 1) with zero bias: h = w^T * x. The decoder reconstructs the input via tied weights: x_hat = w * h. (a) Compute the scalar latent code h, (b) Compute the reconstructed vector x_hat, (c) Calculate the reconstruction error vector e = x - x_hat, and (d) Verify that the reconstruction error vector is orthogonal to the weight vector w (e^T * w = 0).

Given: x = [3.0, 4.0]^T, w = [0.8, 0.6]^T.

Governing Formula: h = w^T * x; x_hat = w * h; e = x - x_hat; Orthogonality check: e^T * w = 0

Step-by-Step Calculation:

Step 1: Compute latent code h: h = (0.8 * 3.0) + (0.6 * 4.0) = 2.4 + 2.4 = 4.80.

Step 2: Compute reconstructed vector x_hat: x_hat = h * w = 4.80 * [0.8, 0.6]^T = [3.84, 2.88]^T.

Step 3: Compute reconstruction error vector e: e = x - x_hat = [3.0 - 3.84, 4.0 - 2.88]^T = [-0.84, 1.12]^T.

Step 4: Compute squared reconstruction loss: Loss = ||e||^2 = (-0.84)^2 + (1.12)^2 = 0.7056 + 1.2544 = 1.960.

Step 5: Verify orthogonality: e^T * w = (-0.84 * 0.8) + (1.12 * 0.6) = -0.672 + 0.672 = 0.0000.

[FINAL EVALUATED ANSWER: h = 4.80, x_hat = [3.84, 2.88]^T, Error e = [-0.84, 1.12]^T (Orthogonal: e^T * w = 0.0000).]

Pedagogical Note: The linear autoencoder performs an exact orthogonal projection onto the 1D principal component subspace, discarding the orthogonal component as reconstruction error.

Real-World Industry Case (Railway Safety & Predictive Infrastructure): Industrial High-Speed Rail Track Flaw Ultrasonic Anomaly Detection

Pipeline Workflow: Ultrasonic inspection cars record rail acoustic waveforms -> Undercomplete Autoencoder trained exclusively on thousands of kilometers of normal, undamaged steel rail -> When a microscopic sub-surface rail crack passes, autoencoder fails to reconstruct it -> High reconstruction error triggers immediate track repair alarm.

Industrial Impact & Benefit: Detects internal rail fatigue fractures before structural derailment failures occur.

Key Oral Viva Voce Checkpoints

Q1: What is an Autoencoder?

Ans: A neural network trained in a self-supervised manner to copy its input to its output through an internal latent representation.

Q2: What is the primary danger of training an unconstrained autoencoder?

Ans: It can learn to perfectly memorize the trivial identity function $g(f(x)) = x$ without extracting useful semantic features.

Q3: What is an Undercomplete Autoencoder?

Ans: An autoencoder whose latent representation dimension k is strictly smaller than the input dimension d.

Q4: What classical statistical technique is equivalent to a linear autoencoder trained with MSE loss?

Ans: Principal Component Analysis (PCA).

Q5: What is a Denoising Autoencoder (DAE)?

Ans: An autoencoder trained to reconstruct the clean original data from an intentionally corrupted or noisy input.

LECTURE #34 · MODULE 5 (Topic 5.7)

Date: 21-Nov-2025
Period: Period 1

Title: Current Research Areas: Representation Learning

CO: CO5 Bloom's: Understand / Analyze

Prerequisites: Autoencoders, Latent variables, Manifolds, Day 33 concepts
Key Terminology: Representation Learning, Manifold Hypothesis, Disentangled Representations, Self-Supervised Learning, Contrastive Learning, InfoNCE Loss, Transfer Learning

Manifold Hypothesis

The Manifold Hypothesis states that real-world high-dimensional data (e.g. 1000x1000 pixel images in R^1,000,000) concentrates its probability mass almost entirely on low-dimensional manifolds embedded within the high-dimensional ambient space. Key Observations: 1. If pixels were uniformly distributed in R^1,000,000, random noise would look like natural scenes. In reality, natural images occupy a tiny fraction of configuration space. 2. Variations in images (lighting, camera pose, object scale, facial expression) correspond to smooth continuous movements along low-dimensional manifold coordinates. 3. Deep networks learn coordinate charts that unfold, unroll, and flatten these complex curved manifolds into Euclidean spaces where linear classifiers succeed.

Disentangled Representations

Bengio, Courville, and Vincent (2013) define an ideal representation as a 'Disentangled Representation'. Properties: - Each coordinate axis of the latent representation h corresponds to an independent real-world explanatory factor of variation (e.g. h_1 = object rotation, h_2 = lighting intensity, h_3 = object identity). - Invariance: A change in lighting modifies only h_2 while leaving identity h_3 completely invariant. - Linear Separability: Complex non-linear decision boundaries on the original data manifold become flat linear hyperplanes in the disentangled representation space.

Self Supervised Learning

Modern Representation Learning utilizes Self-Supervised Learning: 1. Contrastive Learning (SimCLR, Chen et al., 2020): Pulls together representations of augmented views of the same image (positives) while pushing apart representations of different images (negatives) using InfoNCE loss: L_InfoNCE = - ln (exp(sim(z_i, z_j) / tau) / sum_k exp(sim(z_i, z_k) / tau)) 2. Masked Autoencoding (MAE, He et al., 2021): Masks 75% of image patches and trains a Vision Transformer to reconstruct missing pixels, learning rich global representations.

Exam Important Points

- KTU Exam Q20(a): Illustrate the use of representation learning in object classification.
- Definition and physical justification of the Manifold Hypothesis.
- Properties of Disentangled Representations (invariance, linearity, factor independence).
- Formulation of InfoNCE loss in contrastive self-supervised learning.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): InfoNCE Contrastive Loss Calculation

Statement: In a self-supervised contrastive learning step, an anchor representation z_a is compared against 1 positive augmented sample z_p and 2 negative samples {z_n1, z_n2}. Cosine similarities are computed as: sim(z_a, z_p) = 0.80 sim(z_a, z_n1) = 0.20 sim(z_a, z_n2) = -0.10 The temperature parameter is tau = 0.50. Calculate: (a) Exponentiated scaled similarities for all pairs, and (b) The InfoNCE loss value.

Given: sim_pos = 0.80, sim_neg1 = 0.20, sim_neg2 = -0.10, tau = 0.50.

Governing Formula: exp_sim = exp(sim / tau); L = - ln(exp_pos / (exp_pos + sum(exp_neg)))

Step-by-Step Calculation:

Step 1: Compute scaled similarities (sim / tau): pos: 0.80 / 0.50 = 1.60. neg1: 0.20 / 0.50 = 0.40. neg2: -0.10 / 0.50 = -0.20.

Step 2: Compute exponentials: exp_pos = e^(1.60) = 4.9530. exp_neg1 = e^(0.40) = 1.4918. exp_neg2 = e^(-0.20) = 0.8187.

Step 3: Compute denominator sum: Denom = exp_pos + exp_neg1 + exp_neg2 = 4.9530 + 1.4918 + 0.8187 = 7.2635.

Step 4: Compute Softmax Probability: P_pos = 4.9530 / 7.2635 = 0.6819 (68.19%).

Step 5: Compute InfoNCE Loss: L = - ln(0.6819) = 0.3829.

[FINAL EVALUATED ANSWER: P_pos = 0.6819, InfoNCE Loss L = 0.3829]

Pedagogical Note: The model successfully assigned 68.2% of probability mass to the true positive pair, yielding a low contrastive loss of 0.3829.

Problem 2 (Advanced (KTU Exam Level)): Intrinsic Manifold Dimension vs Ambient Pixel Dimension

Statement: An automotive camera produces RGB images of resolution 1024x1024 pixels. The car drives along a single straight highway where camera images vary solely based on 3 physical degrees of freedom: (1) Vehicle forward displacement along road x, (2) Sun elevation angle theta, and (3) Vehicle lane lateral offset y. (a) Calculate the ambient vector space dimension D_ambient, (b) State the intrinsic manifold dimension d_manifold, and (c) Calculate the dimensional compression factor D_ambient / d_manifold.

Given: Image: 1024x1024x3. Physical variations: 3 degrees of freedom.

Governing Formula: D_ambient = H * W * C; d_manifold = Degrees of Freedom; Compression = D_ambient / d_manifold

Step-by-Step Calculation:

Step 1: Compute ambient space dimensionality: D_ambient = 1024 * 1024 * 3 = 1,048,576 * 3 = 3,145,728 dimensions.

Step 2: State intrinsic manifold dimensionality: The underlying physical state space is governed entirely by 3 variables (x, theta, y). Therefore, the intrinsic manifold dimension d_manifold = 3.

Step 3: Compute dimensional ratio: Ratio = 3,145,728 / 3 = 1,048,576x (over 1 million to 1!).

[FINAL EVALUATED ANSWER: D_ambient = 3,145,728. d_manifold = 3. Compression factor = 1,048,576x.]

Pedagogical Note: Despite residing in a 3.1-million-dimensional ambient pixel space, the image data manifold is strictly 3-dimensional. Representation learning isolates these 3 intrinsic axes.

Real-World Industry Case (Digital Pathology & Oncology Diagnostics): Self-Supervised Medical Pathology Slide Pre-Training

Pipeline Workflow: Hospital acquires 1 million unlabeled high-resolution cancer biopsy tissue slides -> Self-Supervised Contrastive Learning (SimCLR/MAE) pre-trains a deep ResNet/ViT backbone -> Fine-tuned on only 50 labeled rare sarcoma slides.

Industrial Impact & Benefit: Achieves 95.2% diagnostic accuracy on ultra-rare cancers where obtaining hundreds of labeled slides is medically impossible.

Key Oral Viva Voce Checkpoints

Q1: What is the Manifold Hypothesis?

Ans: The proposition that high-dimensional real-world data concentrates near low-dimensional continuous sub-manifolds embedded within the high-dimensional ambient space.

Q2: What is a Disentangled Representation?

Ans: A representation where distinct latent coordinates correspond to independent real-world explanatory factors of variation (e.g. azimuth, elevation, shape).

Q3: Why is linear separability a goal of representation learning?

Ans: Because if representation learning maps complex curved manifolds into a space where classes are linearly separable, a simple linear model can achieve optimal classification without overfitting.

Q4: What is Self-Supervised Learning?

Ans: A learning paradigm where supervisory training signals are automatically generated from the data itself (e.g. predicting masked patches, contrastive view invariance) without human labels.

Q5: What is Contrastive Learning?

Ans: A self-supervised approach that pulls representations of positive pairs (augmented views of same sample) together while pushing negative pairs apart.

LECTURE #35 · MODULE 5 (Topic 5.8)

Date: 24-Nov-2025
Period: Period 2

Title: Current Research Areas: Boltzmann Machines and Deep Belief Networks

CO: COS Bloom's: Understand / Analyze

Prerequisites: Probability Theory, Energy-based models, Statistical Physics basics, Day 33 concepts
Key Terminology: Energy-Based Model (EBM), Boltzmann Distribution, Partition Function Z, Restricted Boltzmann Machine (RBM), Bipartite Graph, Gibbs Sampling, Conditional Independence

Energy Based Models

An Energy-Based Model associates a scalar energy E(x) with each configuration of variables. High-probability states correspond to low energy levels. The Boltzmann Distribution defines probability via the Gibbs / Boltzmann distribution: P(x) = exp(- E(x)) / Z where Z = sum_x exp(- E(x)) is the normalizing Partition Function. Intractability: Evaluating Z requires summing over 2^D possible binary states, which is computationally intractable for D > 40.

General Boltzmann Machine

A Boltzmann Machine (1985) is an undirected graphical model containing visible units v (observed data) and hidden units h (latent representations): Energy Function: E(v, h) = - v^T * W * h - (1/2) * v^T * L * v - (1/2) * h^T * J * h - b^T * v - c^T * h Intractable Learning: Because units within the same layer are interconnected (matrices L and J), computing P(h|v) requires extensive Markov Chain Monte Carlo (MCMC) sampling, making training extremely slow.

Restricted Boltzmann Machine

Smolensky (1986) introduced the Restricted Boltzmann Machine (RBM), which forbids intra-layer connections (L = 0 and J = 0). Bipartite Graph Structure: Connections exist strictly between visible units v and hidden units h. Energy Function of RBM: E(v, h) = - v^T * W * h - b^T * v - c^T * h = - sum_i sum_j v_i * W_ij * h_j - sum_i b_i * v_i - sum_j c_j * h_j Conditional Independence: Given visible layer v, all hidden units h_j are mutually independent: P(h_j = 1 | v) = sigma(sum_i W_ij * v_i + c_j) Given hidden layer h, all visible units v_i are mutually independent: P(v_i = 1 | h) = sigma(sum_j W_ij * h_j + b_i) This allows parallel, exact block Gibbs sampling in a single step!

Exam Important Points

- KTU Exam Q20(b): Compare Boltzmann Machine with Deep Belief Network.
- Diagram of general Boltzmann machine (with intra-layer connections) vs RBM (bipartite graph).
- Formulation of RBM energy function E(v, h).
- Derivation of conditional probabilities P(h_j=1|v) and P(v_i=1|h) showing conditional independence.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): RBM Energy and Hidden Activation Probability Calculation

Statement: An RBM has 2 visible units and 2 hidden units. Parameters are: Weight matrix W = [[0.8, -0.4], [0.5, 0.6]] Visible bias b = [0.1, -0.2]^T Hidden bias c = [-0.1, 0.3]^T Given visible vector v = [1, 0]^T: (a) Calculate the activation probabilities P(h_1 = 1 | v) and P(h_2 = 1 | v), and (b) If hidden vector is sampled as h = [1, 1]^T, calculate the energy E(v, h).

Given: v = [1, 0]^T, W = [[0.8, -0.4], [0.5, 0.6]], b = [0.1, -0.2]^T, c = [-0.1, 0.3]^T.

Governing Formula: P(h_j = 1 | v) = sigma(sum_i v_i * W_ij + c_j); E(v, h) = - v^T * W * h - b^T * v - c^T * h

Step-by-Step Calculation:

Step 1: Compute input to hidden units: z_h1 = (v_1 * W_11 + v_2 * W_21) + c_1 = (1 * 0.8 + 0 * 0.5) + (-0.1) = 0.8 - 0.1 = 0.70. z_h2 = (v_1 * W_12 + v_2 * W_22) + c_2 = (1 * -0.4 + 0 * 0.6) + 0.3 = -0.4 + 0.3 = -0.10.

Step 2: Compute conditional probabilities: P(h_1 = 1 | v) = sigma(0.70) = 1 / (1 + e^(-0.70)) = 1 / (1 + 0.49659) = 0.6682. P(h_2 = 1 | v) = sigma(-0.10) = 1 / (1 + e^(0.10)) = 1 / (1 + 1.10517) = 0.4750.

Step 3: Compute Energy for v = [1, 0]^T and h = [1, 1]^T: Term 1: v^T * W = [1, 0] * [[0.8, -0.4], [0.5, 0.6]] = [0.8, -0.4]. (v^T * W) * h = [0.8, -0.4] * [1, 1]^T = 0.8 - 0.4 = 0.40. Term 2: b^T * v = 0.1 * 1 + (-0.2) * 0 = 0.10. Term 3: c^T * h = (-0.1 * 1) + (0.3 * 1) = 0.20. Total Energy E(v, h) = - 0.40 - 0.10 - 0.20 = -0.70.

[FINAL EVALUATED ANSWER: P(h_1=1|v) = 0.6682, P(h_2=1|v) = 0.4750, Energy E(v, h) = -0.70]

Pedagogical Note: The negative energy value (-0.70) indicates a favorable, low-energy configuration under the learned Boltzmann distribution.

Problem 2 (Advanced (KTU Exam Level)): Exact Partition Function and Joint Probability in 2x2 Boltzmann Distribution

Statement: A tiny toy RBM has 1 visible unit v in {0, 1} and 1 hidden unit h in {0, 1}. The energy function is E(v, h) = - 2.0 * v * h. (a) Enumerate all 4 possible joint states (v, h) and compute their energies, (b) Calculate the unnormalized Boltzmann weights exp(-E) for all states, (c) Calculate the exact Partition Function Z, and (d) Calculate the exact joint probability distribution P(v, h).

Given: v in {0, 1}, h in {0, 1}, E(v, h) = - 2.0 * v * h.

Governing Formula: E(v, h) = - Z * v * h; Weight = e^(-E); Z = sum exp(-E); P(v, h) = exp(-E) / Z

Step-by-Step Calculation:

Step 1: Enumerate 4 states and energies: State 1: (0, 0) -> E = -2 * 0 * 0 = 0.0 -> exp(-0) = 1.000. State 2: (0, 1) -> E = -2 * 0 * 1 = 0.0 -> exp(-0) = 1.000. State 3: (1, 0) -> E = -2 * 1 * 0 = 0.0 -> exp(-0) = 1.000. State 4: (1, 1) -> E = -2 * 1 * 1 = -2.0 -> exp(-(-2)) = e^(2.0) = 7.3891.

Step 2: Calculate Partition Function Z: Z = 1.000 + 1.000 + 1.000 + 7.3891 = 10.3891.

Step 3: Calculate joint probabilities: P(0, 0) = 1.0 / 10.3891 = 0.0963 (9.63%) P(0, 1) = 1.0 / 10.3891 = 0.0963 (9.63%) P(1, 0) = 1.0 / 10.3891 = 0.0963 (9.63%) P(1, 1) = 7.3891 / 10.3891 = 0.7112 (71.12%).

[FINAL EVALUATED ANSWER: Partition Function Z = 10.3891. P(0,0)=P(0,1)=P(1,0)=0.0963, P(1,1)=0.7112.]

Pedagogical Note: Because state (1, 1) has the lowest energy (-2.0), the Boltzmann distribution concentrates over 71% of the total probability mass onto that single state.

Real-World Industry Case (Recommender Systems & E-Commerce): Collaborative Filtering Netflix Movie Recommendation

Pipeline Workflow: User movie rating history -> Modeled as visible units of a Conditional RBM -> Hidden units extract latent genre preferences (sci-fi, romance) -> Gibbs sampling reconstructs missing movie ratings.

Industrial Impact & Benefit: Was a core winning algorithmic component of the famous million-dollar Netflix Prize competition (Salakhutdinov et al., 2007).

Key Oral Viva Voce Checkpoints

Q1: What is an Energy-Based Model (EBM)?

Ans: A probabilistic model that defines probability distributions by assigning a scalar energy to configurations, where low energy corresponds to high probability.

Q2: What is the Partition Function Z in a Boltzmann Machine?

Ans: The normalization denominator Z = sum exp(-E(x)) that ensures probabilities sum to 1.0 across all possible configurations.

Q3: Why is training a general Boltzmann Machine computationally intractable?

Ans: Because intra-layer connections prevent conditional independence, making the exact partition function and posterior inference require prohibitive exponential sums.

Q4: What structural restriction defines a Restricted Boltzmann Machine (RBM)?

Ans: No connections are permitted between units within the same layer (no visible-to-visible or hidden-to-hidden connections), forming a bipartite graph.

Q5: What major mathematical benefit arises from the RBM bipartite restriction?

Ans: Conditional independence: all hidden units are conditionally independent given the visible units, and all visible units are conditionally independent given the hidden units.

LECTURE #36 · MODULE 5 (Topic 5.9)

Date: 26-Nov-2025
Period: Period 4

Title: Current Research Areas: Deep Belief Networks (DBNs) and Course Synthesis

CO: CO5 Bloom's: Understand / Evaluate

Prerequisites: Restricted Boltzmann Machines, Energy-Based Models, Day 35 concepts
Key Terminology: Deep Belief Network (DBN), Hybrid Graphical Model, Contrastive Divergence (CD-k), Greedy Layer-Wise Pre-training, Positive Phase, Negative Phase, Fine-Tuning, Course Synthesis

Dbn Architecture

A Deep Belief Network (Hinton et al., 2006) is a deep generative hybrid graphical model formed by stacking multiple Restricted Boltzmann Machines (RBMs). Hybrid Graphical Structure: - Top two layers form an UNDIRECTED bipartite graph (an RBM) that acts as an associative memory. - All lower layers form DIRECTED acyclic generative connections pointing downward from hidden layers to visible layers: P(x, h^1, h^2, ..., h^L) = P(h^{L-1}, h^L) * product_{l=1}^{L-2} P(h^l | h^{l+1}) * P(x | h^1)

Contrastive Divergence

The log-likelihood gradient for an RBM is: del ln P(v) / del W = E_data [v * h^T] - E_model [v * h^T] - Positive Phase (Data-dependent expectation): Easily evaluated using training sample v and P(h|v). - Negative Phase (Model distribution expectation): Requires running an MCMC chain to thermal equilibrium (infinity steps). Contrastive Divergence (CD-k, Hinton 2002): Replaces infinite MCMC sampling by running only k steps of Gibbs sampling (typically k = 1) starting from the data vector v_0: 1. Sample h_0 ~ P(h | v_0) 2. Sample v_1 ~ P(v | h_0) (Reconstruction) 3. Sample h_1 ~ P(h | v_1) Weight update: delta W = eta * (v_0 * h_0^T - v_1 * h_1^T) Astonishingly fast, robust, and revolutionized deep unsupervised training!

Greedy Layerwise Pretraining Protocol

The 2006 breakthrough protocol that launched the deep learning era: 1. Step 1: Train RBM 1 on raw data v to learn feature representation h^1 using CD-1. 2. Step 2: Freeze RBM 1. Use its hidden states h^1 as input data to train RBM 2. 3. Step 3: Repeat greedily layer by layer for L layers. 4. Step 4: Unroll into a feedforward deep network, attach a supervised Softmax classification layer, and perform end-to-end backpropagation Fine-Tuning! Why it worked: Greedy pre-training initialized deep weights into a basin of attraction close to a good local minimum, completely bypassing the vanishing gradient barrier of 2006.

Course Synthesis

Complete KTU CST414 Course Synthesis: - Module 1: Foundations (Perceptrons, MLPs, Backpropagation, Optimization pathologies). - Module 2: Deep Learning Engine (Advanced optimizers: Momentum, NAG, Adam; Regularization: L1/L2, Dropout, Initialization). - Module 3: Spatial Intelligence (CNNs, Pooling, Equivariance, Bottlenecks, Winograd, MobileNet). - Module 4: Temporal & Sequence Intelligence (RNNs, Seq2Seq, BPTT, LSTMs, GRUs). - Module 5: Frontiers & Applications (Vision, Speech, NLP, Word2Vec, GloVe, Autoencoders, DBNs).

Exam Important Points

- Hybrid directed/undirected structure of Deep Belief Networks.
- Contrastive Divergence CD-1 algorithm steps: positive phase vs negative phase.
- Greedy layer-wise unsupervised pre-training and supervised fine-tuning procedure.
- Complete comparison between Boltzmann Machine, RBM, and DBN.

Worked Numerical Problems (Hand-Derived Step-by-Step)

Problem 1 (Basic): Single Step Contrastive Divergence (CD-1) Weight Update Calculation

Statement: An RBM has 2 visible units and 1 hidden unit with scalar weights W = [0.2, -0.1]^T (biases are zero). A training sample is v_0 = [1.0, 1.0]. Learning rate is eta = 0.5. (a) In the positive phase, compute hidden activation probability p_h0, (b) In the negative phase, reconstruct visible vector probabilities p_v1 using hidden activation h_0 = p_h0, and (c) Compute the negative hidden probability p_h1 and the resulting CD-1 weight gradient update delta_W.

Given: v_0 = [1, 1], W = [0.2, -0.1]^T, eta = 0.5.

Governing Formula: p_h0 = sigma(v_0 . W); p_v1 = sigma(h_0 * W^T); p_h1 = sigma(p_v1 . W); delta_W = eta * (v_0^T * p_h0 - p_v1^T * p_h1)

Step-by-Step Calculation:

Step 1: Positive Phase: z_h0 = (1.0 * 0.2) + (1.0 * -0.1) = 0.10. p_h0 = 1 / (1 + e^(-0.10)) = 1 / (1 + 0.90484) = 0.5250. Positive correlation: v_0^T * p_h0 = [1, 1]^T * 0.5250 = [0.5250, 0.5250]^T.

Step 2: Negative Phase (Reconstruction): z_v1_1 = 0.5250 * 0.2 = 0.1050 -> p_v1_1 = sigma(0.1050) = 0.5262. z_v1_2 = 0.5250 * (-0.1) = -0.0525 -> p_v1_2 = sigma(-0.0525) = 0.4869. p_v1 = [0.5262, 0.4869].

Step 3: Negative Hidden Activation: z_h1 = (0.5262 * 0.2) + (0.4869 * -0.1) = 0.1052 - 0.0487 = 0.0565. p_h1 = sigma(0.0565) = 0.5141. Negative correlation: p_v1^T * p_h1 = [0.5262 * 0.5141, 0.4869 * 0.5141]^T = [0.2705, 0.2503]^T.

Step 4: Compute CD-1 Weight Update: delta_W = 0.5 * ([0.5250, 0.5250]^T - [0.2705, 0.2503]^T) = 0.5 * [0.2545, 0.2747]^T = [0.1272, 0.1374]^T.

Step 5: Compute updated weights: W_new = [0.2 + 0.1272, -0.1 + 0.1374]^T = [0.3272, 0.0374]^T.

[FINAL EVALUATED ANSWER: Positive stats = [0.525, 0.525]^T, Negative stats = [0.271, 0.250]^T, Updated W = [0.3272, 0.0374]^T]

Pedagogical Note: Contrastive Divergence pushed weights in the direction of the empirical data distribution and away from the 1-step model hallucination.

Problem 2 (Advanced (KTU Exam Level)): KTU Model Paper Q20(b): Comparative Architectural Analysis of BM vs DBN

Statement: Compare Boltzmann Machines (BM), Restricted Boltzmann Machines (RBM), and Deep Belief Networks (DBN) across: (1) Graph topology and directionality, (2) Parameter count scaling, (3) Training complexity and algorithm, and (4) Generative capability.

Given: Architectures: Boltzmann Machine, RBM, DBN.

Governing Formula: Tabular synthesis and structural contrast.

Step-by-Step Calculation:

Step 1: Graph Topology and Directionality: - Boltzmann Machine: Undirected graph with intra-layer connections within visible and hidden units. - RBM: Undirected bipartite graph with zero intra-layer connections. - DBN: Hybrid model; top two layers are an undirected RBM; all lower layers are directed top-down generative connections.

Step 2: Training Algorithm and Complexity: - Boltzmann Machine: Intractable MCMC requiring infinite Gibbs sampling; exponential complexity O(2^D). - RBM: Contrastive Divergence (CD-k), linear complexity O(k * D_v * D_h). - DBN: Greedy layer-wise unsupervised pre-training using CD-k on stacked RBMs, followed by supervised backpropagation fine-tuning.

Step 3: Inference and Feature Extraction: - Boltzmann Machine: Slow stochastic inference. - RBM: Exact, one-step parallel block Gibbs sampling: P(h|v) in O(1) MCMC steps. - DBN: Feedforward bottom-up inference pass for feature extraction; top-down directed ancestral sampling for generation.

Step 4: Historical Significance: - DBN demonstrated in 2006 that deep networks with many layers could be trained effectively, breaking the 15-year neural network winter.

[FINAL EVALUATED ANSWER: BM: Undirected with intra-layer loops (intractable). RBM: Undirected bipartite (CD-k). DBN: Hybrid directed/undirected (Greedy pre-training + fine-tuning).]

Pedagogical Note: DBNs revolutionized AI by introducing layer-wise pre-training on stacked RBMs, bridging statistical physics with modern deep learning.

Real-World Industry Case (Heavy Industry & Vibration Telemetry Analysis): Industrial Turbine Fault Diagnosis using Deep Belief Networks

Pipeline Workflow: High-dimensional multi-sensor vibration readings from power plant turbines -> 3-layer DBN pre-trained on unlabelled baseline vibrations via CD-1 -> Top layer fine-tuned with 50 labeled bearing failure events -> Detects sub-surface rotor imbalances.

Industrial Impact & Benefit: Enables predictive maintenance that avoids multi-million-dollar turbine rotor catastrophic blade fracture.

Key Oral Viva Voce Checkpoints

Q1: What is a Deep Belief Network (DBN)?

Ans: A deep generative hybrid graphical model consisting of stacked RBMs, where the top two layers form an undirected associative memory and lower layers have directed top-down generative connections.

Q2: Who invented Deep Belief Networks and in what year?

Ans: Geoffrey Hinton, Simon Osindero, and Yee-Whye Teh in 2006.

Q3: What is Contrastive Divergence (CD-k)?

Ans: An approximation algorithm for training RBMs that replaces full MCMC sampling by running only k steps of Gibbs sampling initialized at the data vector.

Q4: What are the two phases of Contrastive Divergence?

Ans: The Positive Phase (measures correlations on true data v0*h0^T) and the Negative Phase (measures correlations on reconstructed data v_k*h_k^T).

Q5: What is Greedy Layer-Wise Unsupervised Pre-training?

Ans: Training each layer of a deep network one by one from bottom to top as an independent RBM on the representations of the previous layer, before unrolling into a supervised network.